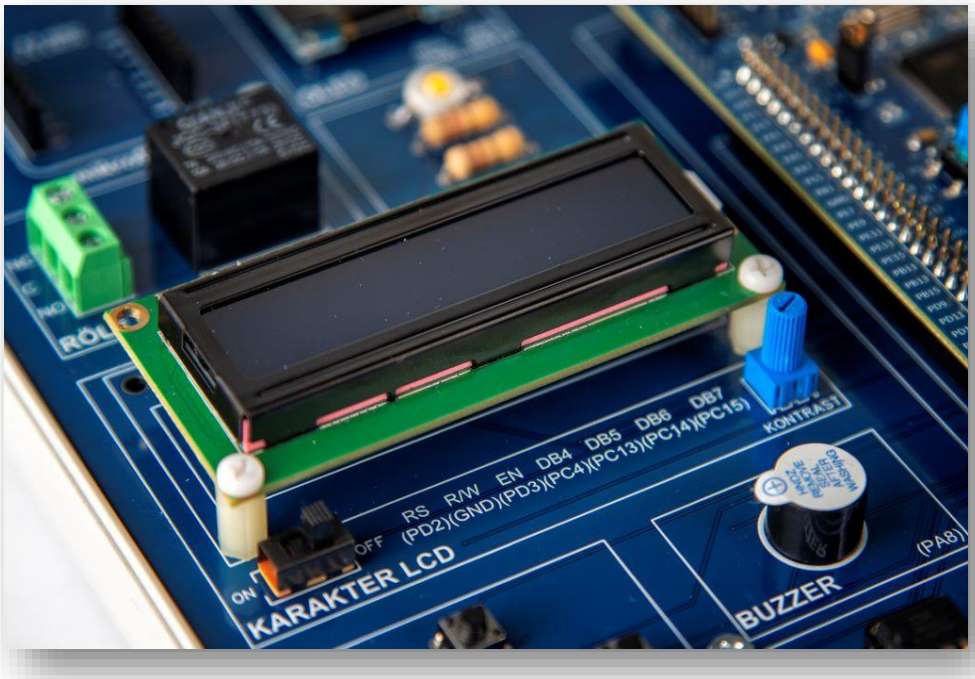
SİVAS CUMHURİYET ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖMÜLÜ SİSTEMLER DERSİ

Gömülü Sistemler Deney 2: Analog-Dijital Dönüştürücü (ADC) Uygulamaları

Potansiyometre Okuma, Voltaj Hesaplama ve LCD Ekran Kullanımı



Dersin Hocası: Doç. Dr. Ahmet Gürkan YÜKSEK
Dersin Asistanı: Arş. Gör. Ahmet Utku ELİK
Öğrenci Adı Soyadı – Numarası:

1. Deneyin Amacı

Bu deneyde, STM32F407 mikrodenetleyicisinin **ADC (Analog-to-Digital Converter)** birimini kullanarak analog sensör verilerinin (Potansiyometre) okunması, dijital veriye dönüştürülmesi ve **16x02 LCD ekran** üzerinde kullanıcıya gösterilmesi amaçlanmaktadır.

2. ADC Nedir?

Mikrodenetleyiciler dijital (0 ve 1) dünyada çalışır, ancak gerçek dünya (ses, sıcaklık, ışık) analogdur. ADC, bu sürekli analog sinyalleri mikrodenetleyicinin anlayabileceği sayısal değerlere dönüştüren köprüdür.

Çözünürlük (Resolution) Ne Demektir?

STM32F407'nin ADC'si 12-bit çözünürlüğe sahiptir.

Bu şu anlama gelir: 0 ile 3.3V arasındaki voltajı, 0 ile 4095 arasında bir sayıya böler.

- **Hassasiyet:** 3.3V / 4096 = 0.0008V. Yani voltajdaki **0.8 milivolt'luk** en ufak bir değişimi bile algılayabiliriz.

3. Kullanılacak Donanım ve Pin Bağlantıları

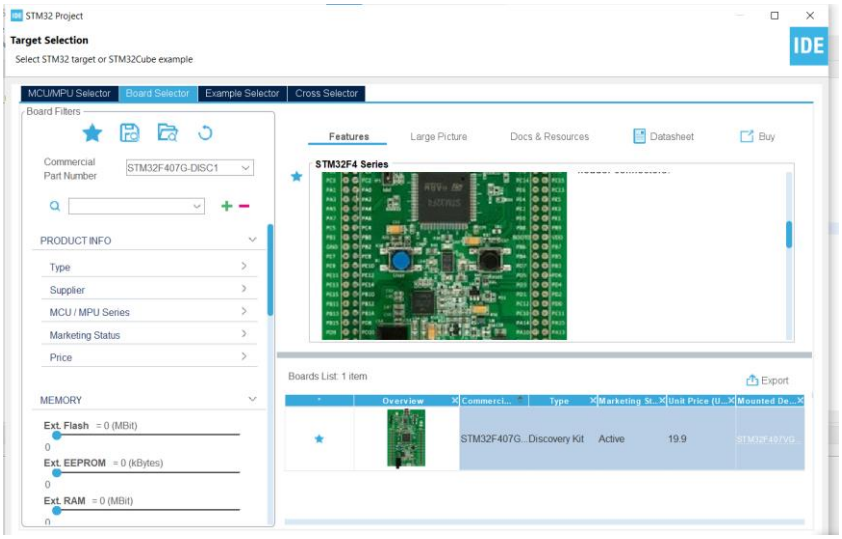
Bileşen	Pin Kodu	Mod	Mod
Potansiyometre	PC5	ADC1_IN15	Analog Giriş (0-3.3V arası)
Switch 0 (SW0)	PE13	GPIO_Input	Voltaj/Ham Veri Seçimi
LCD: RS, EN	PD2, PD3	GPIO_Output	LCD Kontrol Pinleri
LCD: D4-D7	PC4, PC13, PC14, PC15	GPIO_Output	LCD Kontrol Pinleri

4. STM32CubeIDE KONFİGÜRASYONU (CubeMX)

Kodlamaya geçmeden önce mikrodenetleyicinin pinlerini, ADC modülünü ve sistem saatini ayarlamamız gerekmektedir.

Adım 1: Proje Oluşturma

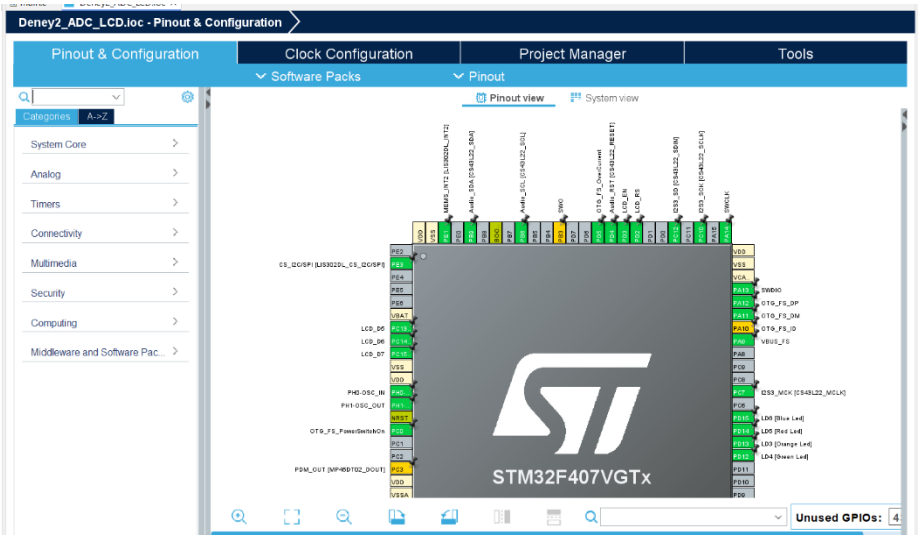
1. STM32CubeIDE'yi açın ve üst menüden File > New > STM32 Project yolunu izleyin.
2. **Part Number** arama kutusuna **STM32F407VG** yazın.
3. Sağdaki listeden işlemciyi seçip **"Next"** butonuna basın.
4. Proje ismine **"Deney2_ADC_LCD"** yazıp **"Finish"** butonuna tıklayın.



Adım 2: Pin Kurulumu ve Etiketleme (Pinout & Configuration)

Sağ taraftaki işlemci görseli üzerinden aşağıdaki pinleri bulup sol tıklayarak ilgili modları seçin ve **sağ tıklayıp "Enter User Label" diyerek** belirtilen etiketleri yazın.

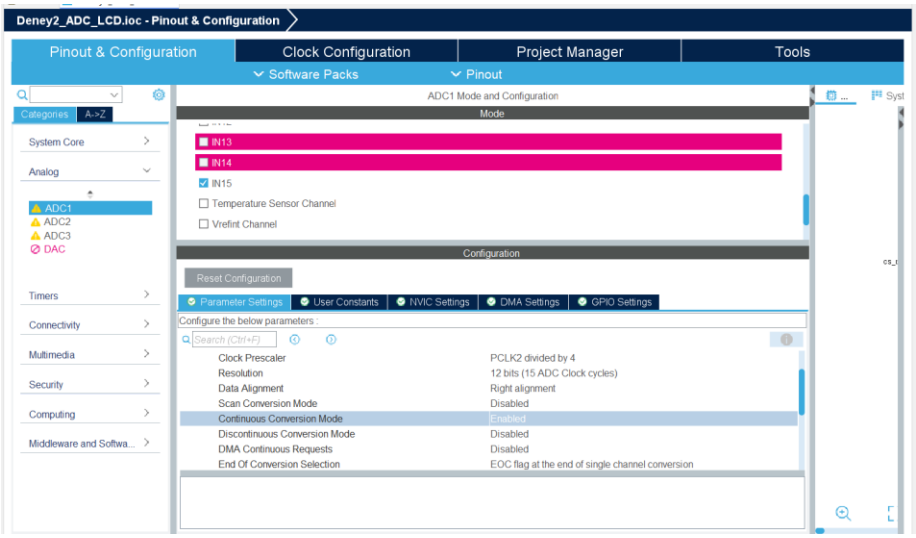
- **Analog Giriş (Potansiyometre):**
  - PC5 -> **ADC1_IN15** seçin. (Etiket: POT)
- **Switch (Buton):**
  - PE13 -> **GPIO_Input** seçin. (Etiket: SW0)
- **LCD Ekran Pinleri (Çıkış):**
  - PD2 -> **GPIO_Output** (Etiket: LCD_RS)
  - PD3 -> **GPIO_Output** (Etiket: LCD_EN)
  - PC4 -> **GPIO_Output** (Etiket: LCD_D4)
  - PC13 -> **GPIO_Output** (Etiket: LCD_D5)
  - PC14 -> **GPIO_Output** (Etiket: LCD_D6)
  - PC15 -> **GPIO_Output** (Etiket: LCD_D7)

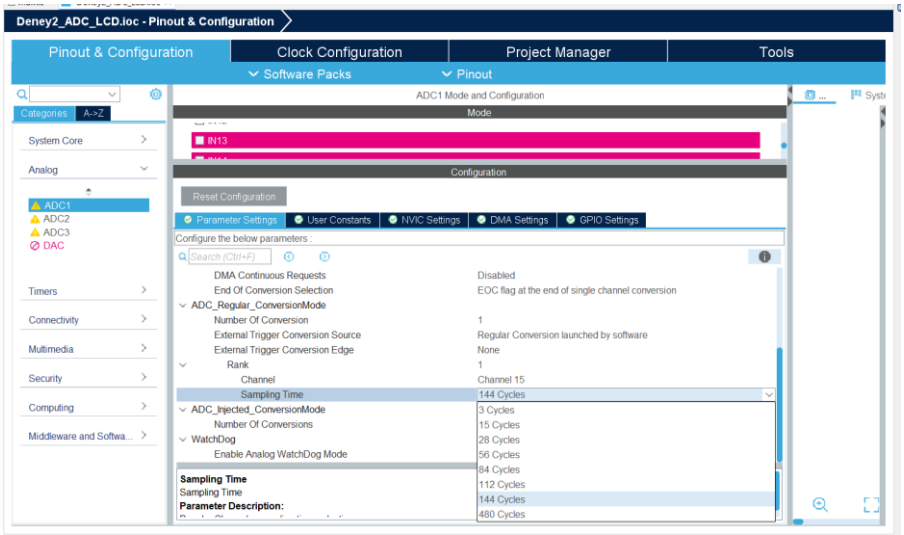


Adım 3: ADC Modülü Ayarları

Potansiyometreyi sürekli ve kararlı okuyabilmek için ADC ayarlarını yapmalıyız.

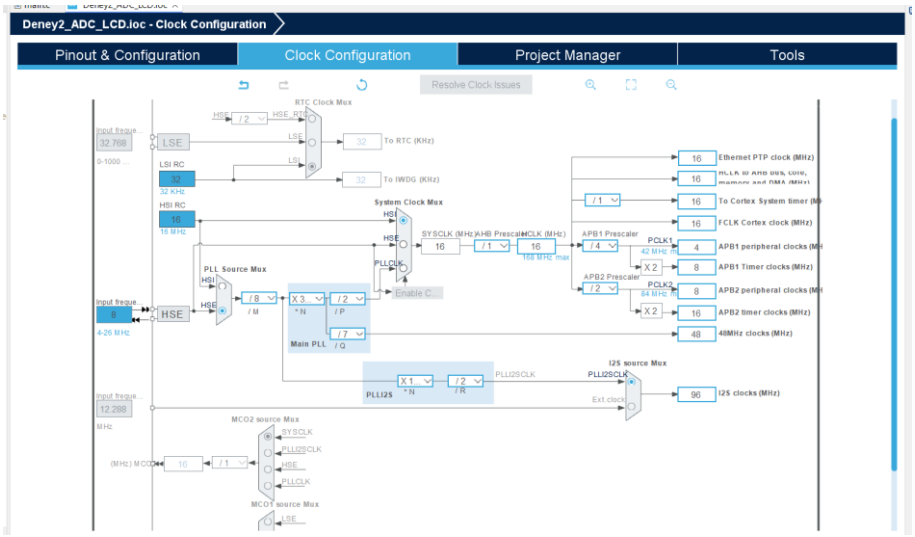
1. Soldaki menüden **Analog** > **ADC1** sekmesine tıklayın.
2. **Mode** kısmında **IN15** kutucuğunu işaretleyin (V).
3. Aşağıdaki **Configuration** penceresinde şu ayarları yapın:
    - Continuous Conversion Mode: **Enabled** (Sürekli okuma modu).
    - Rank başlığını genişletin ve Sampling Time değerini **144 Cycles** yapın (Daha kararlı ölçüm için).





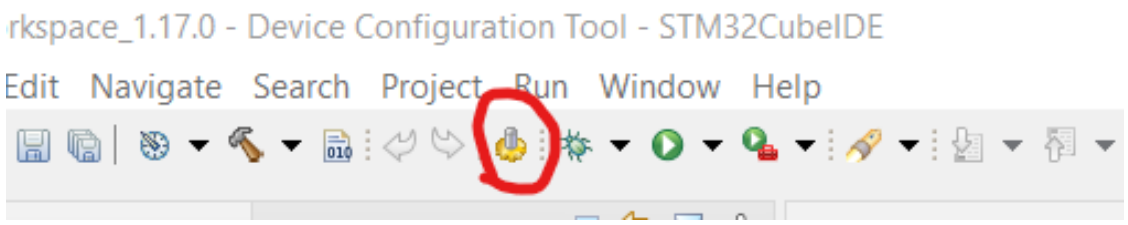
Adım 4: Saat Ayarları (Clock Configuration)

1. Üst sekmelerden **Clock Configuration**'a gelin.
2. Giriş frekansı olarak **HSI (High Speed Internal)** seçili olduğundan emin olun.
3. *Varsayılan 16 MHz hız bu deney için yeterlidir.*



Adım 5: Kod Üretimi (Code Generation)

1. Ayarlar bittikten sonra üst menüdeki **"Device Configuration Tool Code Generation"** (Sarı dişli simgesi ⚙) butonuna basın.
2. **"Code is successfully generated"** uyarısını aldığınızda main.c dosyanız açılacaktır.



5. Yazılım ve Algoritma

5.1. LCD Kütüphanesi ve Gerekli Kütüphanelerin Eklenmesi

STM32 HAL kütüphanesinde standart bir karakter LCD desteği bulunmamaktadır. Bu projede LCD ekranı kontrol edebilmek için harici bir sürücü kütüphanesi oluşturulmalıdır.

Bunun için proje klasöründe manuel olarak **lcd.h** ve **lcd.c** dosyaları oluşturulup, içerisine aşağıdaki kodlar yazılmalıdır.

5.1.1. lcd.h Dosyasının Oluşturulması

Bu dosya, fonksiyon prototiplerini içerir ve projenin **Core/Inc** klasörü altına kaydedilmelidir.

Dosya İçeriği (lcd.h):

```
#ifndef INC_LCD_H_
#define INC_LCD_H_

#include "stm32f4xx_hal.h"

// Fonksiyon Prototipleri
void LCD_Init(void);                      // LCD'yi başlatma
void LCD_SendCommand(uint8_t cmd);        // Komut gönderme
void LCD_SendData(uint8_t data);          // Veri (Karakter) gönderme
void LCD_Clear(void);                     // Ekranı temizleme
void LCD_SetCursor(uint8_t row, uint8_t col); // İmleç konumlandırma
void LCD_Print(char *str);                // String (Metin) yazdırma

#endif /* INC_LCD_H_ */
```

5.1.2. lcd.c Dosyasının Oluşturulması

Bu dosya, LCD'nin nasıl çalışacağını belirleyen asıl kodları içerir ve projenin **Core/Src** klasörü altına kaydedilmelidir. Kodlar, CubeMX'te tanımlanan pin etiketlerini (User Label) kullanır.

Dosya İçeriği (lcd.c):

```
#include "lcd.h"
#include "main.h" // Pin tanımlarını buradan alır (LCD_RS_Pin, LCD_EN_Pin vb.)

// Yardımcı Fonksiyon: 4-Bit Veri Gönder
void LCD_Send4Bits(uint8_t data) {
    HAL_GPIO_WritePin(LCD_D4_GPIO_Port, LCD_D4_Pin, (data & 1) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D5_GPIO_Port, LCD_D5_Pin, ((data >> 1) & 1) ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D6_GPIO_Port, LCD_D6_Pin, ((data >> 2) & 1) ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LCD_D7_GPIO_Port, LCD_D7_Pin, ((data >> 3) & 1) ?
GPIO_PIN_SET : GPIO_PIN_RESET);
}

// Komut Gönderme Fonksiyonu
void LCD_SendCommand(uint8_t cmd) {
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_RESET); // RS = 0
(Komut)

    LCD_Send4Bits(cmd >> 4); // Üst 4 bit
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(1);

    LCD_Send4Bits(cmd & 0x0F); // Alt 4 bit
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(2);
}

// Veri Gönderme Fonksiyonu
void LCD_SendData(uint8_t data) {
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_SET); // RS = 1
(Veri)

    LCD_Send4Bits(data >> 4);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(1);

    LCD_Send4Bits(data & 0x0F);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_SET);
    HAL_Delay(1);
    HAL_GPIO_WritePin(LCD_EN_GPIO_Port, LCD_EN_Pin, GPIO_PIN_RESET);
```

```
    HAL_Delay(1);
}

// LCD Başlatma (4-Bit Modu)
void LCD_Init(void) {
    HAL_Delay(50);
    LCD_SendCommand(0x02); // 4-bit moda geçiş
    LCD_SendCommand(0x28); // 2 satır, 5x8 font
    LCD_SendCommand(0x0C); // Ekran Açık
    LCD_SendCommand(0x06); // Sağa kaydır
    LCD_SendCommand(0x01); // Temizle
    HAL_Delay(2);
}

void LCD_Clear(void) {
    LCD_SendCommand(0x01);
    HAL_Delay(2);
}

void LCD_SetCursor(uint8_t row, uint8_t col) {
    uint8_t address;
    if (row == 0) address = 0x80 + col;
    else address = 0xC0 + col;
    LCD_SendCommand(address);
}

void LCD_Print(char *str) {
    while (*str) {
        LCD_SendData(*str++);
    }
}
```

5.2. Voltaj Dönüşüm Formülü ADC biriminden okunan değer **12-bitlik** ham veridir (0 - 4095). Bu değeri anlamlı bir gerilim (Volt) değerine dönüştürmek için şu formül kullanılır:

$$\text{Voltaj} = \frac{\text{Okunan ADC} * 3.3}{4095}$$

5.3. Ana Yazılım Kodu (main.c)
Projenin main.c dosyasına öncelikle gerekli kütüphaneler eklenmeli ve ardından algoritma kodlanmalıdır.

Kütüphane Ekleme (Includes Bölümü):

```
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "lcd.h"     // Oluşturduğumuz LCD kütüphanesi
#include <stdio.h>   // sprintf fonksiyonu için gerekli
```

Değişken Tanımlamaları (USER CODE BEGIN 0):

```
/* USER CODE BEGIN 0 */
uint16_t adc_ham_deger = 0; // ADC'den okunan ham veri
float voltaj_degeri = 0.0;  // Hesaplanmış voltaj değeri
char lcd_buffer[16];        // LCD metin tamponu
/* USER CODE END 0 */
```

Başlangıç ve Sonsuz Döngü (main fonksiyonu):

```
/* USER CODE BEGIN 2 */
  LCD_Init();            // LCD başlat
  HAL_ADC_Start(&hadc1); // ADC başlat (Sürekli mod)

  // --- AÇILIŞ SENARYOSU ---
  LCD_SetCursor(0, 0);
  LCD_Print("SCU BIL MUH");
  HAL_Delay(2000);
  LCD_Clear();

  LCD_SetCursor(0, 0);
  LCD_Print("DENEY-2 ADC");
  LCD_SetCursor(1, 0);
  LCD_Print("Grup No: 1");
  HAL_Delay(3000);
  LCD_Clear();
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
      // ADC Okuma
      if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
      {
          adc_ham_deger = HAL_ADC_GetValue(&hadc1);
      }

      // Satır 1: Ham Değer
      LCD_SetCursor(0, 0);
      sprintf(lcd_buffer, "ADC Val: %4d", adc_ham_deger);
      LCD_Print(lcd_buffer);

      // Satır 2: Voltaj veya Boşluk (Switch Kontrolü)
      if (HAL_GPIO_ReadPin(SW0_GPIO_Port, SW0_Pin) == GPIO_PIN_SET)
      {
          voltaj_degeri = (float)adc_ham_deger * 3.3 / 4095.0;
          LCD_SetCursor(1, 0);
          sprintf(lcd_buffer, "ADC (V): %.2f V", voltaj_degeri);
          LCD_Print(lcd_buffer);
      }
      else
      {
          LCD_SetCursor(1, 0);
          LCD_Print("                "); // Satırı temizle
      }

      HAL_Delay(100);
```

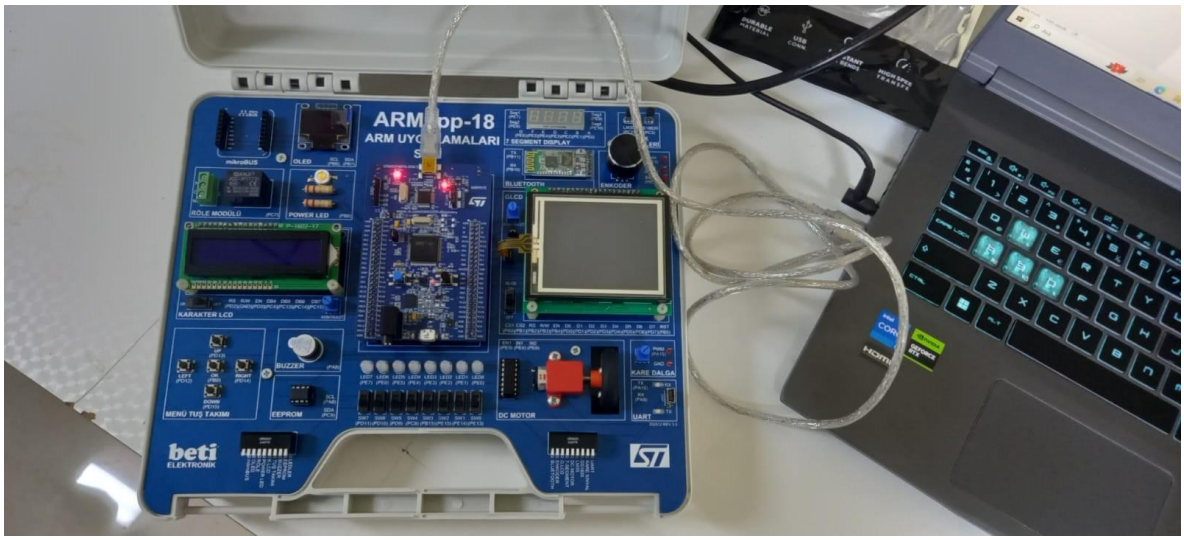
```
  }
  /* USER CODE END WHILE */
```

5. Pratik Uygulama ve Test Adımları

Yazdığımız kodun STM32 kartına yüklenmesi ve deney seti üzerinde test edilmesi için aşağıdaki adımları izleyin.

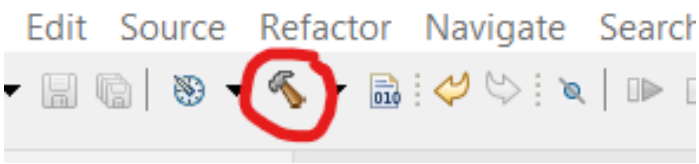
Adım 1: Fiziksel Bağlantı

1. ArmApp18 deney seti üzerindeki **STM32F4 Discovery** kartının üst kısmında bulunan **mini-USB (USB ST-LINK)** girişine kabloyu takın.
2. Kablonun diğer ucunu bilgisayarınızın USB portuna bağlayın.
3. Kart üzerindeki **PWR** (Güç) LED'inin yandığını ve bilgisayarınızın donanımı tanıdığını doğrulayın.

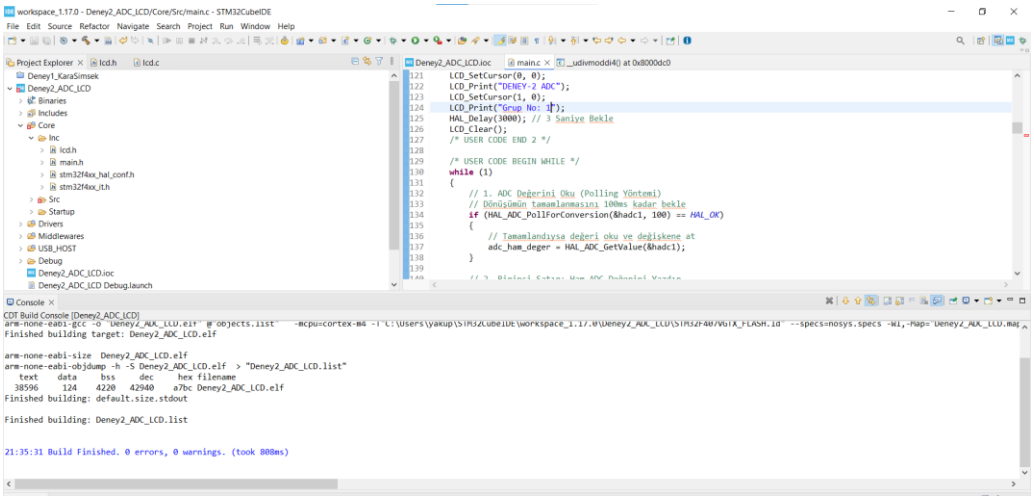


Adım 2: Derleme ve Yükleme (Build & Run)

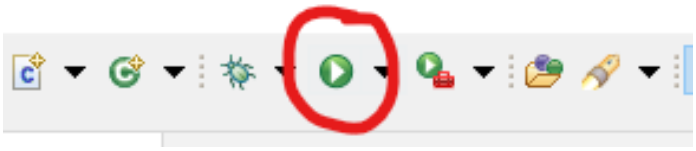
1. **Derleme:** STM32CubeIDE üst menüsündeki **"Çekiç" (Build)** ikonuna tıklayın (veya Ctrl+B).



- *Console penceresinde "0 errors, 0 warnings" mesajını görmelisiniz.*

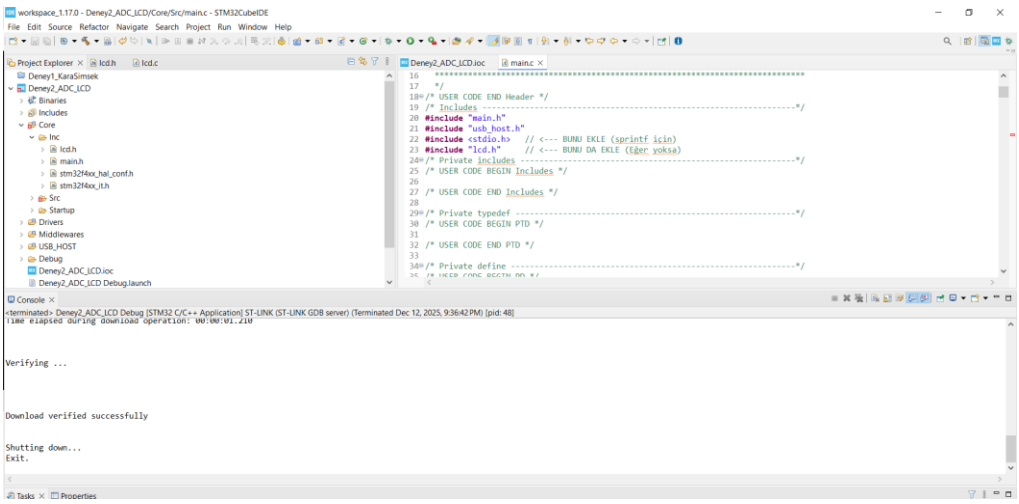


2. **Yükleme:** Derleme hatasız ise, yeşil **"Oynat" (Run)** ikonuna tıklayın.



- *İlk yüklemede "Edit Configuration" penceresi açılırsa "OK" diyerek geçin.*
- *ST-LINK Firmware uyarısı gelirse "Upgrade" işlemini yapın.*

3. **Onay:** Console penceresinde **"Download verified successfully"** mesajını gördüğünüzde kod karta yüklenmiş demektir.



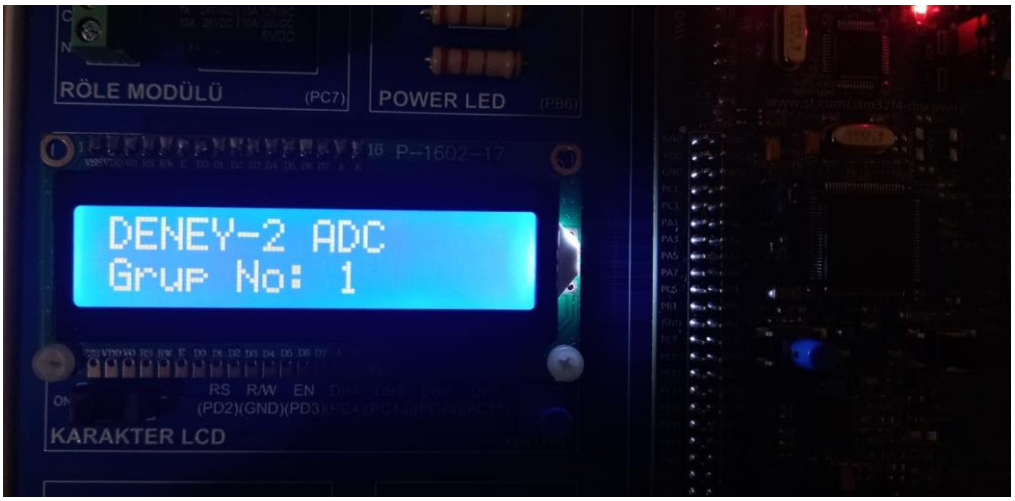
6. Sonuç ve Değerlendirme

Bu laboratuvar çalışmasında, **STM32F407VGT6** mikrodenetleyicisi ve ArmApp18 eğitim seti kullanılarak **ADC (Analog-Dijital Dönüştürücü)** modülünün çalışma prensipleri ve **LCD (Sıvı Kristal Ekran)** kontrolü uygulamalı olarak incelenmiştir.

Deney Sürecinde Gerçekleştirilen Aşamalar:

1. **Donanım Konfigürasyonu:** STM32CubeMX arayüzü ile **ADC1** birimi sürekli dönüşüm (Continuous Conversion) modunda, **GPIO** pinleri ise LCD ve buton kontrolü için uygun modlarda yapılandırılmıştır.
2. **Yazılım Entegrasyonu:** Standart HAL kütüphanesinde bulunmayan LCD sürücü kodları (lcd.c ve lcd.h) projeye dahil edilmiş ve başarıyla kullanılmıştır.
3. **Veri İşleme:** Potansiyometreden okunan 12-bitlik (0-4095) ham analog veri, matematiksel formüllerle (V = ADC * 3.3 / 4095) gerilim değerine dönüştürülmüştür.
4. **Kullanıcı Etkileşimi:** Sisteme eklenen buton (SW0) kontrolü ile ekranın çalışma modu dinamik olarak değiştirilmiş; kullanıcı isteğine göre ham veri veya voltaj değeri gösterilmiştir.

Elde Edilen Sonuçlar: Yapılan testler sonucunda; açılış animasyonlarının ("SCU BIL MUH", "Grup No") doğru çalıştığı, potansiyometre değişimlerinin ekrana anlık yansıdığı ve buton kontrolünün işlevini hatasız yerine getirdiği gözlemlenmiştir. Bu deney ile gömülü sistemlerde sensör verilerinin okunması ve insan-makine arayüzü (HMI) oluşturulması konularında yetkinlik kazanılmıştır.


RÖLE MODÜLÜ
(PC7)
POWER LED
(PB6)
P-1602-17
DENEY-2 ADC
Grup No: 1
RS R/W EN
(PD2)(GND)(PD3)
KARAKTER LCD

RÖLE MODÜLÜ
(PC7)
POWER LED
(PB6)
P-1602-17
ADC Val: 4095
RS R/W EN DB4
(PD2)(GND)(PD3)(PC4)
KARAKTER LCD